

TROUBLESHOOTING GUIDE

Automated Deployment System for React Application

Prepared by: Akshaya

This guide lists the issues encountered while building and running the CI/CD pipeline for the React application (GitHub → Jenkins → Docker Hub → AWS EC2), along with their causes, solutions, and how each fix was verified. If any item below did not occur in your own setup, treat it as a 'Possible Problem' to watch for rather than something that definitely happened.

1. Docker command not found on EC2

Problem: Docker commands did not work on the EC2 instance because Docker was not installed or the Docker service was not running.

Solution:

```
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
```

Verification:

```
docker --version
systemctl status docker
```

2. Permission denied while running Docker

Problem: The ec2-user account could not run Docker commands without sudo.

Cause: The user account was not added to the docker group.

Solution:

```
sudo usermod -aG docker ec2-user
```

Log out and log in again for the group membership to take effect.

Verification:

```
docker ps
```

3. Docker container name already exists

Problem: Docker displayed an error because the container name react-container was already in use.

Cause: An old container with the same name already existed from a previous run.

Solution:

```
docker stop react-container || true
docker rm react-container || true
```

Then run the new container.

Verification:

```
docker ps
```

4. React application was not accessible through the browser

Problem: The React application could not be opened using the EC2 public IP.

Possible causes:

- Port 80 was not allowed in the EC2 security group.
- The Docker container was not running.
- The port mapping was incorrect.
- The EC2 instance was stopped.

Solution:

Check that the container is running and review its logs:

```
docker ps
docker logs react-container
```

Confirm the security group allows inbound traffic on port 80:

```
Type: HTTP    Port: 80    Source: 0.0.0.0/0
```

Verification:

Open the application in a browser using the EC2 public IP:

```
http://65.0.32.37
```

5. Docker image could not be pulled from Docker Hub

Problem: The EC2 instance could not download the Docker image.

Possible causes:

- Incorrect Docker Hub repository name.
- Incorrect image tag.
- Network connection problem.
- Private Docker Hub repository accessed without logging in.

Solution:

Verify the exact image name and tag being pulled:

```
docker pull akshayanimuthu/react-app:dev
```

Verification:

Confirm the image is present locally:

```
docker images
```

6. Jenkins could not push the image to Docker Hub

Problem: The Jenkins pipeline failed during the Docker Hub push stage.

Possible causes:

- Incorrect Docker Hub username.
- Invalid or expired Docker Hub access token.
- Incorrect Jenkins credential ID.
- Incorrect Docker Hub repository name.

Solution:

Check the Jenkins credential used for the push stage:

```
dockerhub-credentials
```

- Use a Docker Hub access token instead of your account password.
- Confirm the image is tagged and pushed as shown below.

```
akshayamanimuthu/react-app:dev
```

Verification:

Confirm that the dev tag appears in the Docker Hub repository.

7. Jenkins could not connect to EC2 through SSH

Problem: The deployment stage failed because Jenkins could not connect to the EC2 instance.

Possible causes:

- Incorrect EC2 public IP address.
- Incorrect or missing SSH key.
- Port 22 was blocked in the security group.
- The SSH Agent plugin was not installed in Jenkins.
- The SSH public key was not added to the EC2 instance's `authorized_keys` file.

Solution:

Check each of the following in turn:

- EC2 public IP address: 65.0.32.37
- Port 22 is allowed in the EC2 security group.
- Jenkins SSH credential ID: ec2-ssh-key
- The public key exists in `/home/ec2-user/.ssh/authorized_keys` on the instance.

Verification:

Test the SSH connection manually:

```
ssh ec2-user@65.0.32.37
```

8. Jenkins pipeline failed during the Docker build stage

Problem: Jenkins failed while building the Docker image.

Possible causes:

- The Dockerfile was missing from the repository.
- The build folder (React production build) was missing.
- The Dockerfile referenced an incorrect path.
- Docker was not running on the Jenkins machine.

Solution:

Confirm that the repository contains a valid Dockerfile, for example:

```
FROM nginx:alpine
COPY build /usr/share/nginx/html
```

Verification:

Confirm Docker is available and running on the Jenkins machine:

```
docker --version
docker ps
```

9. The old application continued running after deployment

Problem: The updated React application was not visible after a new deployment.

Cause: The old Docker container was still running and was not replaced by the new one.

Solution:

Stop and remove the old container, then pull and run the latest image:

```
docker stop react-container || true
docker rm react-container || true
docker pull akshayanimuthu/react-app:dev
docker run -d --name react-container -p 80:3000 \
  --restart unless-stopped \
  akshayanimuthu/react-app:dev
```

Verification:

Refresh the browser and check the application again.

10. The EC2 public IP address changed

Problem: The application could not be accessed using the previously known IP address.

Cause: The EC2 instance received a new public IP address after being stopped and started.

Solution:

Check the current public IPv4 address in the AWS EC2 console and update the Jenkinsfile and README if necessary.

Prevention: Use an AWS Elastic IP to keep the public address stable across restarts.

Troubleshooting Summary

The issues encountered while building this pipeline mainly involved Docker installation and permissions, container name conflicts, EC2 security group rules, Docker Hub authentication, and Jenkins-to-EC2 SSH connectivity. Each issue was resolved by checking service status, verifying image names and tags, validating Jenkins credentials, and confirming security group rules, resulting in a stable and repeatable deployment pipeline.